Architecture note

What I prioritized and why

The brief asks for depth in a few areas instead of breadth. I put the depth where a shared-document product breaks first in real use:

1. **Access control that can't be bypassed.** All permission rules live in one pure module (`src/lib/access.ts`). Every data function in `src/lib/documents.ts` calls it before reading or writing. API routes never check roles themselves. Because the rules are pure functions, the whole permission matrix is unit-tested. Users with no access get a 404 instead of a 403, so the app doesn't reveal that a document exists.
2. **Not losing people's work, in either direction.** Autosave plus optimistic concurrency. Each document has a `version`, and a save is a conditional `UPDATE … WHERE version = baseVersion`. If someone else saved first, the server returns 409 with who saved. The editor never overwrites the other person, and it never throws away this user's edits either: it loads the latest version in place (no page reload) and keeps the unsaved edits in a read-only panel beside it. From there the user can copy from them, explicitly choose "Use my version", or discard them. Renames don't bump the version, so they never cause conflicts. A first version of this simply asked the user to reload, which was correct but lost their recent typing, so I changed it.
3. **Formatting that survives.** Documents are stored as TipTap (ProseMirror) JSON, not HTML. The server validates every save against the same editor schema the browser uses, so a broken or hand-crafted request can't store a document the editor can't open. One shared extension list in `src/lib/editor-extensions.ts` guarantees the browser and server agree.
4. **Safe links.** Link targets are restricted to http, https and mailto in three places: the editor, the importer, and server-side validation of every save. The schema check alone doesn't inspect attribute values, so a `javascript:` link posted straight to the API would otherwise be stored.
5. **A file import that's useful and safe.** `.md`, `.txt` and `.docx` are each converted to HTML and then parsed through the editor schema. Anything the editor doesn't support, including scripts and event handlers, is dropped by construction instead of by a hand-written sanitizer.

Key decisions

Decision	Why	Tradeoff
Next.js (one app for the UI and the API)	One deploy, one language, server components load data without a client fetch	API routes are tied to Next. Fine at this scope.

Decision	Why	Tradeoff
TipTap	Mature ProseMirror wrapper, schema-based, works on the server for import and validation	Adds about 130 kB to the editor page
Postgres (Neon) + Drizzle	Real relational data for shares. Free hosted tier, so reviewers pay nothing. Drizzle is pure JavaScript with SQL migrations and no binary engine.	SQLite would be simpler locally, but Vercel's filesystem isn't persistent
Mock auth with an HMAC-signed, httpOnly cookie	The brief allows seeded users. Signing the cookie means a user id can't be forged by editing it.	No passwords or real identity. Swapping in a real identity provider only changes `src/lib/session.ts`. Testers can also create their own demo account (name and email, no password); the three seeded accounts always stay listed first, and the number of accounts is capped so a public demo can't grow without bound.
Owner + editor/viewer roles	One step beyond basic access (an optional stretch item), and it exercises the permission design	No commenter role, no link sharing
Import creates a new document	The clearest product flow ("bring my existing doc in")	No attachments or importing into an existing draft
Markdown export on the server; PDF through print	Markdown reuses the stored JSON and is tested by a round trip through the importer. Browser print gives a good PDF with no headless browser on the server.	PDF quality depends on the browser's print dialog
Conflict recovery instead of live merging	Keeps both versions and lets the person decide; a few hundred lines instead of a CRDT	Merging is manual (copy and paste, or replace)

Data model

```
users(id, email unique, name, created_at)
documents(id, title, content jsonb, owner_id → users, version, last_edited_by_id →
users, created_at, updated_at)
shares(document_id → documents, user_id → users, role editor|viewer)   PK(document_id,
user_id)
```

Deleting a document cascades to its shares. Re-sharing with the same person updates their role instead of adding a duplicate.

Deliberately left out

- **Real-time co-editing (CRDT/Yjs) and presence.** This is the largest piece of a real Google Docs, and it can't be done well in the timebox. The version check gives correctness (no silent overwrites) without it.
- **Real authentication**, password reset and organizations.
- **Comments, suggestion mode, version history.**
- **Images, tables and code blocks.**
- **Search and pagination of the document list.** The user directory is listed openly for the demo. A real product would search users instead.

What I'd build next (2–4 hours)

1. Live "Bob just saved" notices (polling or server-sent events), so an open editor can pick up changes before a conflict ever happens.
2. Version history: keep a snapshot per save (or every N minutes) with restore. This is cheap because content is already JSON with a version number.
3. Playwright end-to-end tests in CI for sharing and conflicts (these flows were scripted and run in headless Chrome during development, but not committed as a suite).
4. Real authentication behind the existing `session.ts` interface.